PES UNIVERSITY

Department of Computer Science & Engineering

Object Oriented Analysis and Design

UE23CS352B

Assignment 10 Submission

Name of the Student	Pranav Hemanth
SRN	PES1UG23CS433
Section	G
Department	CSE
Campus	RR

Object Oriented Analysis and Design (UE23CS352B) - Lab 10

Problems on Design Patterns

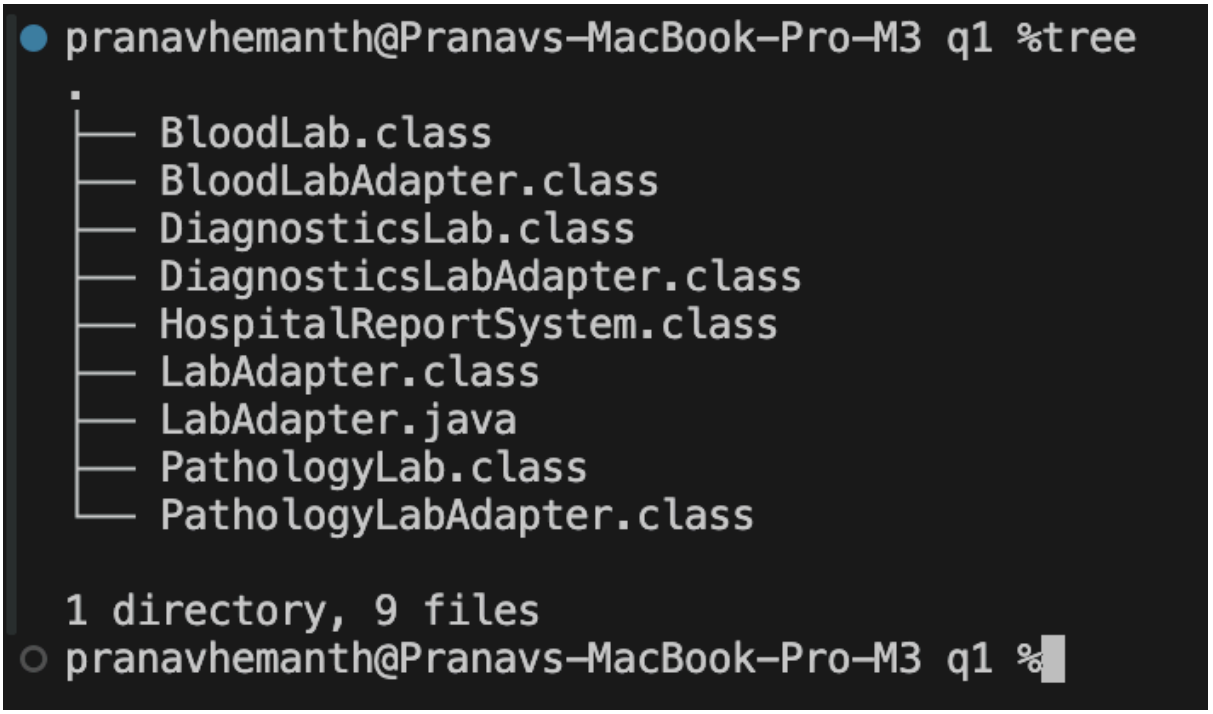
Department of Computer Science & Engineering
Object Oriented Analysis and Design

UE23CS352B

Problem Statements

1. A hospital management system integrates patient data from multiple external labs. Each lab provides reports using different methods such as getBloodReport(), fetchLabResults(), and retrieveDiagnostics(), while the hospital system expects a unified method getPatientReport(). The developers plan to introduce a component that converts these different method calls into a single standard format without modifying existing lab classes. Identify and implement the appropriate design pattern and explain how it resolves interface mismatches.

Source files list:



Project structure for Q1 showing all source files

LabAdapter.java code:

Lab10 > q1 > LabAdapter.java > HospitalReportSystem

```java
// === Q1: ADAPTER PATTERN ===
// Target interface expected by the hospital system
interface HospitalReportSystem {
    String getPatientReport();
}

// Existing lab classes (Adaptees) - cannot be modified
class BloodLab {
    public String getBloodReport() {
        return "Blood Lab Report: Hemoglobin=13.5, WBC=7000, Platelets=250000";
    }
}

class DiagnosticsLab {
    public String fetchLabResults() {
        return "Diagnostics Lab Results: X-Ray=Normal, MRI=Clear";
    }
}

class PathologyLab {
    public String retrieveDiagnostics() {
        return "Pathology Report: Biopsy=Benign, Culture=Negative";
    }
}

// Adapters - wrap each lab to match HospitalReportSystem interface
class BloodLabAdapter implements HospitalReportSystem {
    private BloodLab bloodLab;

    public BloodLabAdapter(BloodLab bloodLab) {
        this.bloodLab = bloodLab;
    }

    @Override
    public String getPatientReport() {
        return "[ADAPTED] " + bloodLab.getBloodReport();
    }
}

```

```java
class DiagnosticsLabAdapter implements HospitalReportSystem {
    private DiagnosticsLab diagnosticsLab;

    public DiagnosticsLabAdapter(DiagnosticsLab diagnosticsLab) {
        this.diagnosticsLab = diagnosticsLab;
    }

    @Override
    public String getPatientReport() {
        return "[ADAPTED] " + diagnosticsLab.fetchLabResults();
    }
}

class PathologyLabAdapter implements HospitalReportSystem {
    private PathologyLab pathologyLab;

    public PathologyLabAdapter(PathologyLab pathologyLab) {
        this.pathologyLab = pathologyLab;
    }

    @Override
    public String getPatientReport() {
        return "[ADAPTED] " + pathologyLab.retrieveDiagnostics();
    }
}

// Client - Hospital system uses only getPatientReport()
public class LabAdapter {
    Run | Debug
    public static void main(String[] args) {
        HospitalReportSystem[] labs = {
            new BloodLabAdapter(new BloodLab()),
            new DiagnosticsLabAdapter(new DiagnosticsLab()),
            new PathologyLabAdapter(new PathologyLab())
        };

        System.out.println(x: "=== Hospital Patient Reports ===");
        for (HospitalReportSystem lab : labs) {
            System.out.println(lab.getPatientReport());
        }
    }
}
```

Output:

```
pranavhemanth@Pranavs-MacBook-Pro-M3 q1 %java LabAdapter
=== Hospital Patient Reports ===
[ADAPTED] Blood Lab Report: Hemoglobin=13.5, WBC=7000, Platelets=250000
[ADAPTED] Diagnostics Lab Results: X-Ray=Normal, MRI=Clear
[ADAPTED] Pathology Report: Biopsy=Benign, Culture=Negative
pranavhemanth@Pranavs-MacBook-Pro-M3 q1 %
```

The Adapter pattern resolves interface mismatches between incompatible classes. Here, each external lab exposes a different method (getBloodReport(), fetchLabResults(), retrieveDiagnostics()), but the hospital system expects a uniform getPatientReport(). Adapter classes wrap each lab and translate the call, so the client interacts with one standard interface without modifying any existing lab code.

2. An online video streaming platform loads high-resolution videos using a class with methods like loadVideo(String name) and playVideo(). Since loading videos is resource-intensive, the system currently delays playback due to repeated loading of the same content. Developers want to introduce an intermediary that controls access and ensures videos are loaded only when needed and reused when requested again. Identify and implement the suitable design pattern and explain how it improves performance.

Source files list:

```
pranavhemanth@Pranavs-MacBook-Pro-M3 q2 %tree
.
├── RealVideo.class
├── Video.class
├── VideoProxyDemo.class
├── VideoProxyDemo.java
└── VideoProxyHandler.class

1 directory, 5 files
pranavhemanth@Pranavs-MacBook-Pro-M3 q2 %
```

Project structure for Q2 showing all source files

VideoProxyDemo.java code:

Lab10 > q2 > VideoProxyDemo.java > Video

```java
// === Q2: PROXY PATTERN (Virtual Proxy + Caching) ===

// Subject interface
interface Video {
    void loadVideo(String name);
    void playVideo();
}

// RealSubject - expensive to create
class RealVideo implements Video {
    private String name;

    public RealVideo(String name) {
        this.name = name;
        loadVideo(name); // loading happens at construction
    }

    @Override
    public void loadVideo(String name) {
        System.out.println("Loading high-resolution video from server: " + name);
        // Simulate expensive load
        try { Thread.sleep(millis: 500); } catch (InterruptedException e) {}
        System.out.println("Video loaded: " + name);
    }

    @Override
    public void playVideo() {
        System.out.println("Playing video: " + name);
    }
}

// Proxy - controls access, lazy loads, and caches
class VideoProxyHandler implements Video {
    private RealVideo realVideo;
    private String name;
    private static java.util.Map<String, RealVideo> cache = new java.util.HashMap<>();

    public VideoProxyHandler(String name) {
        this.name = name;
    }

```

```java
    @Override
    public void loadVideo(String name) {
        // Lazy loading: only load when play is actually called
        if (!cache.containsKey(name)) {
            System.out.println("[PROXY] Cache miss. Initiating load for: " + name);
            cache.put(name, new RealVideo(name));
        } else {
            System.out.println("[PROXY] Cache hit. Reusing loaded video: " + name);
        }
        realVideo = cache.get(name);
    }

    @Override
    public void playVideo() {
        loadVideo(name); // lazy load on first play
        realVideo.playVideo();
    }
}

public class VideoProxyDemo {
    Run | Debug
    public static void main(String[] args) {
        System.out.println(x: "=== Video Streaming with Proxy ===\n");

        Video v1 = new VideoProxyHandler(name: "Inception.mp4");
        Video v2 = new VideoProxyHandler(name: "Interstellar.mp4");
        Video v3 = new VideoProxyHandler(name: "Inception.mp4"); // same as v1 - should use cache

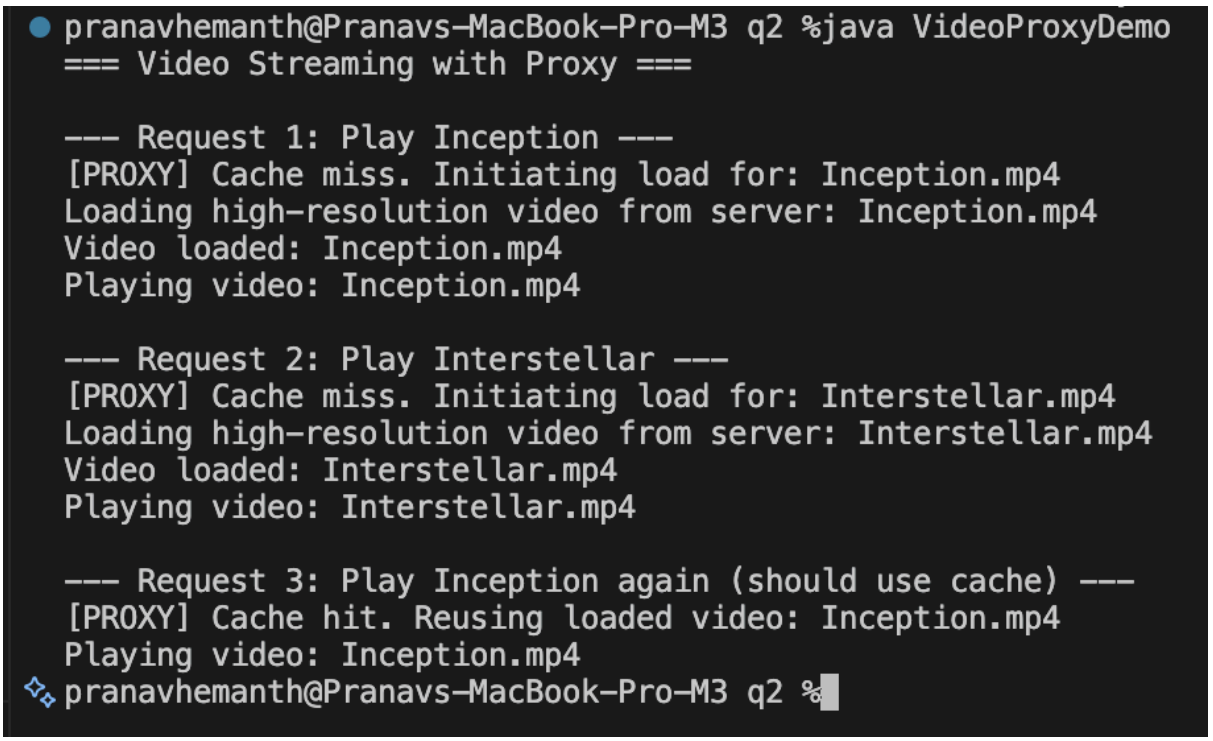
        System.out.println(x: "--- Request 1: Play Inception ---");
        v1.playVideo();

        System.out.println(x: "\n--- Request 2: Play Interstellar ---");
        v2.playVideo();

        System.out.println(x: "\n--- Request 3: Play Inception again (should use cache) ---");
        v3.playVideo();
    }
}

```

Output:



The Proxy pattern controls access to an expensive object. VideoProxyHandler wraps RealVideo and defers loading until playVideo() is actually called (lazy loading). A static cache ensures the same video is loaded from the server only once and subsequent requests reuse the cached instance. This reduces redundant server calls and improves response time.

3. A gaming application displays thousands of identical trees, rocks, and buildings across different locations in a virtual environment. Each object currently stores properties like texture, color, and type along with position coordinates, resulting in high memory usage. Developers want to reuse common properties and store only unique positional data separately. Identify and implement the design pattern that best fits this requirement and explain how it reduces memory consumption

Source files list:

```
pranavhemanth@Pranavs-MacBook-Pro-M3 q3 %tree
.
├── FlyweightGame.class
├── FlyweightGame.java
├── GameObjectFactory.class
├── GameObjectFlyweight.class
├── Tree.class
└── TreeType.class

1 directory, 6 files
pranavhemanth@Pranavs-MacBook-Pro-M3 q3 %
```

Project structure for Q3 showing all source files

FlyWeightGame.java code:

Lab10 > q3 > J FlyweightGame.java > ...

```java
// === Q3: FLYWEIGHT PATTERN ===
// Separates intrinsic (shared) state from extrinsic (unique) state

import java.util.HashMap;
import java.util.Map;

// Flyweight interface
interface GameObjectFlyweight {
    void render(int x, int y); // x,y = extrinsic (positional) state
}

// ConcreteFlyweight - stores intrinsic (shared) state: type, texture, color
class TreeType implements GameObjectFlyweight {
    private String type;
    private String texture;
    private String color;

    public TreeType(String type, String texture, String color) {
        this.type = type;
        this.texture = texture;
        this.color = color;
        System.out.println("[FLYWEIGHT CREATED] Type=" + type + ", Texture=" + texture + ", Color=" + color);
    }

    @Override
    public void render(int x, int y) {
        System.out.println("Rendering " + type + " [texture=" + texture + ", color=" + color + "] at (" + x + ", " + y + ")");
    }
}

// Flyweight Factory - returns existing flyweight or creates new one
class GameObjectFactory {
    private static Map<String, TreeType> flyweights = new HashMap<>();

    public static TreeType getTreeType(String type, String texture, String color) {
        String key = type + "_" + texture + "_" + color;
        if (!flyweights.containsKey(key)) {
            flyweights.put(key, new TreeType(type, texture, color));
        } else {
            System.out.println("[REUSING FLYWEIGHT] " + key);
        }
        return flyweights.get(key);
    }

    public static int getFlyweightCount() {
        return flyweights.size();
    }
}

```

```java
// Context - stores extrinsic state (position) and reference to flyweight
class Tree {
    private int x, y;
    private GameObjectFlyweight flyweight;

    public Tree(int x, int y, String type, String texture, String color) {
        this.x = x;
        this.y = y;
        this.flyweight = GameObjectFactory.getTreeType(type, texture, color);
    }

    public void render() {
        flyweight.render(x, y);
    }
}

public class FlyweightGame {
    Run | Debug
    public static void main(String[] args) {
        System.out.println(x: "=== Game Environment - Flyweight Pattern ===\n");

        // Place 6 trees, only 2 unique types needed
        Tree[] forest = {
            new Tree(x: 10, y: 20, type: "Oak",  texture: "bark.png",  color: "Green"),
            new Tree(x: 30, y: 50, type: "Pine", texture: "pine.png",  color: "DarkGreen"),
            new Tree(x: 55, y: 15, type: "Oak",  texture: "bark.png",  color: "Green"),   // reuses Oak flyweight
            new Tree(x: 70, y: 90, type: "Pine", texture: "pine.png",  color: "DarkGreen"), // reuses Pine flyweight
            new Tree(x: 12, y: 80, type: "Oak",  texture: "bark.png",  color: "Green"),   // reuses Oak flyweight
            new Tree(x: 44, y: 33, type: "Pine", texture: "pine.png",  color: "DarkGreen"), // reuses Pine flyweight
        };

        System.out.println(x: "\n--- Rendering Forest ---");
        for (Tree t : forest) {
            t.render();
        }

        System.out.println("\nTotal trees rendered : " + forest.length);
        System.out.println("Unique flyweight objects in memory: " + GameObjectFactory.getFlyweightCount());
        System.out.println("Memory saved: " + (forest.length - GameObjectFactory.getFlyweightCount()) + " objects avoided");
    }
}
```

Output:

```
pranavhemanth@Pranavs-MacBook-Pro-M3 q3 %java FlyweightGame
=== Game Environment - Flyweight Pattern ===

[FLYWEIGHT CREATED] Type=Oak, Texture=bark.png, Color=Green
[FLYWEIGHT CREATED] Type=Pine, Texture=pine.png, Color=DarkGreen
[REUSING FLYWEIGHT] Oak_bark.png_Green
[REUSING FLYWEIGHT] Pine_pine.png_DarkGreen
[REUSING FLYWEIGHT] Oak_bark.png_Green
[REUSING FLYWEIGHT] Pine_pine.png_DarkGreen

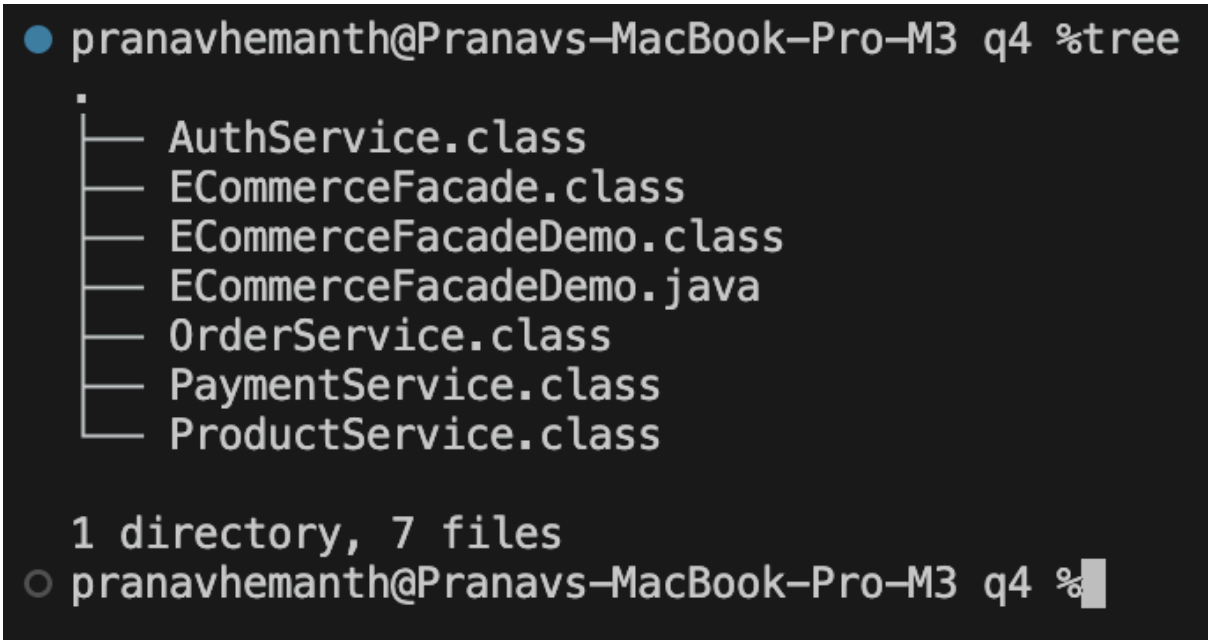
--- Rendering Forest ---
Rendering Oak [texture=bark.png, color=Green] at (10, 20)
Rendering Pine [texture=pine.png, color=DarkGreen] at (30, 50)
Rendering Oak [texture=bark.png, color=Green] at (55, 15)
Rendering Pine [texture=pine.png, color=DarkGreen] at (70, 90)
Rendering Oak [texture=bark.png, color=Green] at (12, 80)
Rendering Pine [texture=pine.png, color=DarkGreen] at (44, 33)

Total trees rendered : 6
Unique flyweight objects in memory: 2
Memory saved: 4 objects avoided
pranavhemanth@Pranavs-MacBook-Pro-M3 q3 %
```

The Flyweight pattern reduces memory by sharing common data across many objects. Intrinsic state (type, texture, color) is stored once in a shared TreeType object; extrinsic state (x, y position) is kept per instance in Tree. The factory returns existing flyweights instead of creating duplicates. Here, 6 trees use only 2 flyweight objects, at scale this eliminates thousands of redundant allocations.

4. An e-commerce platform requires customers to go through multiple steps such as login(), selectProduct(), makePayment(), and trackOrder(). Currently, users must call each method individually, making the process complex. Developers want to provide a single method completePurchase() that internally coordinates all these steps. Identify and implement the design pattern used and explain how it simplifies system interaction.

Source files list:



Project structure for Q4 showing all source files

ECommerceFacadeDemo.java code:

Lab10 > q4 > ECommerceFacadeDemo.java > AuthService

```java
// === Q4: FACADE PATTERN ===
// Provides a single simplified interface to a complex subsystem

// Subsystem classes - complex, many methods
class AuthService {
    public void login(String user) {
        System.out.println("[AuthService] User logged in: " + user);
    }
    public void logout(String user) {
        System.out.println("[AuthService] User logged out: " + user);
    }
}

class ProductService {
    public void selectProduct(String product) {
        System.out.println("[ProductService] Product selected: " + product);
    }
    public void checkStock(String product) {
        System.out.println("[ProductService] Stock available for: " + product);
    }
}

class PaymentService {
    public void makePayment(double amount) {
        System.out.println("[PaymentService] Payment of Rs." + amount + " processed successfully.");
    }
    public void sendInvoice(String user) {
        System.out.println("[PaymentService] Invoice sent to: " + user);
    }
}

class OrderService {
    public void placeOrder(String product) {
        System.out.println("[OrderService] Order placed for: " + product);
    }
    public void trackOrder(String orderId) {
        System.out.println("[OrderService] Tracking order: " + orderId + " - Status: Shipped");
    }
}

```

```java
// Facade - hides complexity behind one simple method
class ECommerceFacade {
    private AuthService auth;
    private ProductService product;
    private PaymentService payment;
    private OrderService order;

    public ECommerceFacade() {
        this.auth = new AuthService();
        this.product = new ProductService();
        this.payment = new PaymentService();
        this.order = new OrderService();
    }

    // Single entry point - coordinates all subsystems internally
    public void completePurchase(String user, String productName, double amount) {
        System.out.println("--- Starting Purchase for " + user + " ---");
        auth.login(user);
        product.checkStock(productName);
        product.selectProduct(productName);
        payment.makePayment(amount);
        payment.sendInvoice(user);
        order.placeOrder(productName);
        order.trackOrder("ORD-" + System.currentTimeMillis() % 10000);
        auth.logout(user);
        System.out.println(x: "--- Purchase Complete ---\n");
    }
}

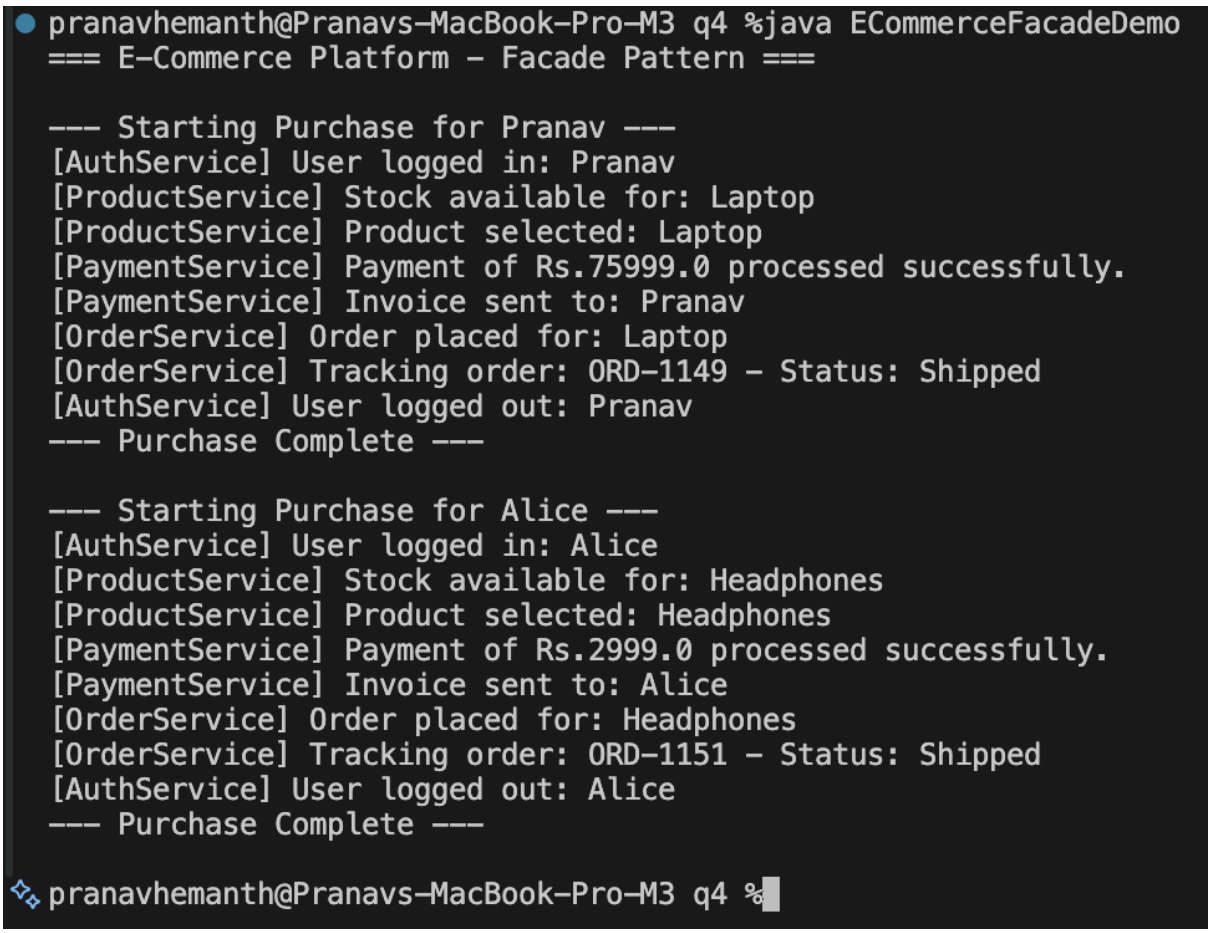
// Client - uses only the facade, unaware of subsystem complexity
public class ECommerceFacadeDemo {
    Run | Debug
    public static void main(String[] args) {
        System.out.println(x: "=== E-Commerce Platform - Facade Pattern ===\n");

        ECommerceFacade facade = new ECommerceFacade();

        facade.completePurchase(user: "Pranav", productName: "Laptop", amount: 75999.00);
        facade.completePurchase(user: "Alice",  productName: "Headphones", amount: 2999.00);
    }
}

```

Output:



The Facade pattern hides subsystem complexity behind a single simplified interface. Instead of calling login(), checkStock(), makePayment(), placeOrder(), and trackOrder() individually, the client calls one method: completePurchase(). ECommerceFacade internally coordinates all four services in the correct order. This reduces client coupling and centralizes workflow logic.